TransitOps SRS

Software Requirements Specification (SRS)

TransitOps — Fleet & Logistics Operations Platform

Version: 1.0 **Prepared for:** Odoo Hackathon 2026 **Conforms to (informally):** IEEE 830 structure **Companion documents:** PRD, TRD, Backend Schema, ER Diagram, UML Class Diagram, Trip Lifecycle Workflow, UI/UX Theme Guide

---

1. Introduction

1.1 Purpose

This SRS specifies the functional and non-functional requirements for TransitOps, a role-based fleet and logistics operations web application. It is intended for the development team, QA, and evaluators to verify that the implemented system meets its intended behavior.

1.2 Scope

TransitOps allows an organization to manage vehicles, drivers, trips, fuel logs, expenses, maintenance records, and derived cost/performance reports through a REST API consumed by a browser-based frontend. The system enforces role-based access control and a set of operational business rules (e.g. no overloaded vehicles, no double-booked assets, no expired-license dispatch).

In scope: vehicle/driver master data, trip lifecycle management, fuel & expense logging, maintenance tracking, reporting/analytics, notifications, authentication/authorization, audit logging.

Out of scope (current release): live GPS telematics, payroll processing, customer-facing booking, native mobile apps, production-grade AI route optimization (route optimizer and AI assistant endpoints exist as unimplemented stubs).

1.3 Definitions, Acronyms, Abbreviations

Term	Meaning
RBAC	Role-Based Access Control
JWT	JSON Web Token
BR-n	Business Rule identifier, as implemented in `tripService.js` / `maintenanceController.js`
KPI	Key Performance Indicator
PII	Personally Identifiable Information
CRUD	Create, Read, Update, Delete

1.4 References

- TransitOps PRD v1.0
- TransitOps TRD v1.0
- TransitOps Backend Schema v1.0
- Source repository: `transitops/backend`, `transitops/frontend`

1.5 Overview

Section 2 describes the product context and user classes. Section 3 lists specific functional and non-functional requirements. Section 4 covers external interface requirements. Section 5 lists system features mapped to use cases. Section 6 covers data requirements. Section 7 lists constraints and assumptions.

---

2. Overall Description

2.1 Product Perspective

TransitOps is a standalone, self-contained web application (Node.js/Express REST API + MongoDB + vanilla-JS frontend), independent of any external ERP at this time, though its Odoo Hackathon origin implies a future integration path.

2.2 Product Functions (summary)

- User authentication and role-based authorization
- Vehicle master data management
- Driver master data management (with encrypted PII)
- Trip lifecycle management (draft → dispatch → in-transit → complete/cancel)
- Fuel log and expense recording
- Maintenance record management
- Dashboard KPIs and cost/performance reporting, incl. CSV export
- System-generated and user-facing notifications
- Audit logging of trip state changes

2.3 User Classes and Characteristics

Role	Characteristics	Primary Use
Fleet Manager	Full oversight; technical/operational authority	Vehicle CRUD, all reports, driver-performance report, CSV export
Dispatcher	Operational, time-sensitive tasks	Trip creation, dispatch, completion, cancellation
Safety Officer	Compliance-focused	Driver CRUD, maintenance record creation/closure
Financial Analyst	Cost/finance-focused	Fuel & expense entry, cost reports, CSV export
Viewer	Read-only stakeholder	Dashboard/report viewing only

2.4 Operating Environment

- Server: Node.js 18 (Alpine in Docker), Express 4.19, MongoDB 6.0
- Client: any modern evergreen browser (no build step required for the frontend)
- Deployment: Docker Compose (dev/small-scale) or Kubernetes (Deployment + Service manifests)

2.5 Design and Implementation Constraints

- MongoDB document schema (not relational) — see Backend Schema doc for full field/validation detail.
- Single-process in-memory cache (`node-cache`) — not safe across multiple horizontally-scaled instances without further work (see TRD §6).
- Frontend has no build step (vanilla JS) — limits the use of a component framework/bundler-driven architecture.

2.6 Assumptions and Dependencies

- A MongoDB instance is reachable via `MONGO_URI`.
- `JWT_SECRET` is set (the server refuses to start otherwise).
- Users are provisioned with exactly one role; the system does not currently support multi-role users.

---

3. Specific Requirements

3.1 Functional Requirements

3.1.1 Authentication & Authorization

- **FR-AUTH-01:** The system shall allow a user to register with name, email, phone, password (≥8 chars), and an

optional role.
- **FR-AUTH-02:** The system shall allow a registered user to log in with email + password and receive a JWT access token.
- **FR-AUTH-03:** The system shall support token refresh via a dedicated endpoint.
- **FR-AUTH-04:** The system shall support a forgot-password / reset-password flow via emailed token (nodemailer).
- **FR-AUTH-05:** The system shall reject any request to a protected endpoint that lacks a valid, non-expired JWT (HTTP 401).
- **FR-AUTH-06:** The system shall reject any request to a role-restricted endpoint made by a user whose role is not in the endpoint's allow-list (HTTP 403).

3.1.2 Vehicle Management

- **FR-VEH-01:** The system shall allow a `fleet_manager` to create, update, and delete vehicle records.
- **FR-VEH-02:** The system shall allow any authenticated user to list and view vehicle records.
- **FR-VEH-03:** The system shall enforce a unique `registrationNumber` per vehicle.
- **FR-VEH-04:** The system shall track vehicle `status` (`available` , `on_trip` , `in_shop` , `retired`) and update it automatically as a side effect of trip dispatch/completion and maintenance open/close.

3.1.3 Driver Management

- **FR-DRV-01:** The system shall allow a `safety_officer` to create, update, and delete driver records.
- **FR-DRV-02:** The system shall allow any authenticated user to list and view driver records.
- **FR-DRV-03:** The system shall enforce a unique `licenseNumber` per driver.
- **FR-DRV-04:** The system shall encrypt `licenseNumber` and `contactNumber` at rest.
- **FR-DRV-05:** The system shall track driver `status` (`available` , `on_trip` , `off_duty` , `suspended`) and license expiry, both of which gate trip assignment eligibility.

3.1.4 Trip Lifecycle

- **FR-TRP-01:** The system shall allow an authorized dispatch-class user to create a trip in `draft` status with source, destination, vehicle, driver, cargo weight, and schedule window.
- **FR-TRP-02:** The system shall reject trip creation if the vehicle is missing or `retired` (BR-1).
- **FR-TRP-03:** The system shall reject trip creation if the driver is missing, `suspended` , or has an expired license (BR-4).
- **FR-TRP-04:** The system shall reject trip creation if cargo weight exceeds the vehicle's `maxLoadCapacity` (BR-5).
- **FR-TRP-05:** The system shall reject trip creation or edit if the requested schedule window overlaps another active trip's window for the same vehicle or driver, and shall identify the conflicting trip in the error response.
- **FR-TRP-06:** The system shall allow editing of a trip's details only while it is in `draft` status.
- **FR-TRP-07:** The system shall allow dispatch of a `draft` trip only if the vehicle and driver are currently `available` , the driver's license is valid, and cargo still fits capacity; on dispatch, it shall set both vehicle and driver to `on_trip` .
- **FR-TRP-08:** The system shall allow completion of a trip only from `dispatched` , `in_transit` , or `delivered` status; on completion it shall set the vehicle and driver back to `available` , increment the vehicle's odometer by the recorded actual distance, and stamp `completedAt` .
- **FR-TRP-09:** The system shall generate a "Maintenance Due Soon" notification automatically when a completed trip brings a vehicle's odometer within 500 km of its `nextMaintenanceMileage` .
- **FR-TRP-10:** The system shall allow cancellation of a trip only from `draft` or `dispatched` status, releasing the vehicle/driver back to `available` if they had been reserved.
- **FR-TRP-11:** The system shall permanently lock a trip's status once it reaches `completed` or `cancelled` (no further transitions).
- **FR-TRP-12:** The system shall support filtering the trip list by status, priority, vehicle, driver, date range, and free-text trip-code search, with pagination.
- **FR-TRP-13:** The system shall write an audit log entry for every trip create, dispatch, complete, cancel, and status-change action.

3.1.5 Fuel & Expense Tracking

- **FR-FUEL-01:** The system shall allow `financial_analyst` and `fleet_manager` users to record fuel logs (liters, cost, station, optional trip link).
- **FR-FUEL-02:** The system shall allow the same roles to record operational expenses categorized by type (toll, insurance, tax, fine, parking, other).
- **FR-FUEL-03:** The system shall allow any authenticated user to view fuel logs and expenses.
- **FR-FUEL-04:** The system shall expose a live fuel-rate reference endpoint.

3.1.6 Maintenance Management

- **FR-MNT-01:** The system shall allow `fleet_manager` or `safety_officer` users to open a maintenance record against a vehicle.
- **FR-MNT-02:** The system shall reject opening a maintenance record if the vehicle is currently `on_trip` or `retired` (BR-9).
- **FR-MNT-03:** The system shall allow updating an open maintenance record.
- **FR-MNT-04:** The system shall allow closing a maintenance record, and shall reject the operation if the record is already closed.

3.1.7 Reporting

- **FR-RPT-01:** The system shall provide a dashboard KPI endpoint summarizing fleet/trip/cost metrics.
- **FR-RPT-02:** The system shall provide a per-vehicle fuel-efficiency report.
- **FR-RPT-03:** The system shall provide a per-vehicle operational-cost report combining fuel, expenses, maintenance cost, and revenue.
- **FR-RPT-04:** The system shall provide a driver-performance report, restricted to `financial_analyst` and `fleet_manager`.
- **FR-RPT-05:** The system shall provide a CSV export of trip data, restricted to `financial_analyst` and `fleet_manager`.
- **FR-RPT-06:** List/report GET endpoints shall be cached for 300 seconds and invalidated on relevant writes.

3.1.8 Notifications

- **FR-NOT-01:** The system shall allow a user to list their notifications.
- **FR-NOT-02:** The system shall allow a user to mark a notification as read.
- **FR-NOT-03:** The system shall support system-wide broadcast notifications (no `userId`).

3.2 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Security	All protected endpoints require a valid JWT; role-restricted endpoints enforce RBAC allow-lists.
NFR-02	Security	Passwords are stored only as bcrypt hashes (cost factor 12), never in plaintext.
NFR-03	Security	Driver license number and contact number are encrypted at rest (AES-256-CBC).
NFR-04	Security	All request bodies are sanitized against NoSQL injection and XSS (`express-mongo-sanitize` , `xss-clean`).
NFR-05	Security	API requests are rate-limited to 100 requests per 10 minutes per IP on `/api/*` .
NFR-06	Security	JSON request bodies are capped at 10kb.
NFR-07	Reliability	The server shall refuse to start if `JWT_SECRET` is not configured.
NFR-08	Reliability	The server shall shut down gracefully on SIGTERM/SIGINT, closing HTTP and DB connections before exit.
NFR-09	Performance	Cached list/report GET responses shall be served from an in-process cache with a default TTL of 300 seconds.
NFR-10	Performance	Response payloads shall be compressed (`compression` middleware).
NFR-11	Availability	The production container shall run under PM2 in cluster mode, restarting on crash.
NFR-12	Auditability	All trip lifecycle state changes shall be

		recorded in an immutable audit log with actor, action, and payload.
NFR-13	Observability	All requests/errors shall be logged via Winston to both file (`combined.log` , `error.log`) and console (dev only).
NFR-14	Usability	Every entity status shall be represented with both color and a text label / icon (not color alone) in the UI.
NFR-15	Portability	The backend shall be containerized (Docker) and deployable to Kubernetes without code changes.
NFR-16	Data Integrity	Concurrent updates to the same document shall be guarded by optimistic concurrency control (version-based).
NFR-17	Data Retention	Deletion of Vehicle, Driver, Trip, FuelLog, Expense, and Maintenance records shall be a soft delete (data retained, flagged), not a hard delete.

---

4. External Interface Requirements

4.1 User Interfaces

Browser-based UI (HTML/CSS/JS) with pages for: Login, Dashboard, Trips, Vehicles, Drivers, Fuel & Expense, Maintenance, Reports, Analytics, Tracking (placeholder), and a shared error page. Visual design follows the dark glassmorphism theme defined in the UI/UX Theme Guide (amber primary accent, status-coded badges, Poppins/Inter typography).

4.2 API / Software Interfaces

REST/JSON API under `/api/v1`, documented via Swagger UI at `/api-docs`. Full endpoint list is in the TRD §4.2. Authentication via `Authorization: Bearer <JWT>` header.

4.3 Communication Interfaces

- HTTPS expected at the reverse-proxy/ingress layer (Nginx in the reference deployment); the app itself sends HSTS headers via Helmet.
- CORS restricted to configured frontend origin(s) in production.
- Outbound email via SMTP (nodemailer) for password-reset flows.

4.4 Hardware Interfaces

None — the system has no direct hardware interface (no telematics/GPS device integration in the current release).

---

5. System Features (Use-Case Summary)

Use Case	Primary Actor	Preconditions	Postconditions
UC-1: Create & Dispatch a Trip	Dispatcher	Vehicle available, driver available & licensed, no schedule conflict	Trip in `dispatched` ; vehicle & driver `on_trip`
UC-2: Complete a Trip	Dispatcher	Trip in an active status	Trip `completed` ; vehicle/driver freed; odometer updated; maintenance notice if due
UC-3: Cancel a Trip	Dispatcher	Trip in `draft` or `dispatched`	Trip `cancelled` ; assets freed if reserved
UC-4: Register a Vehicle	Fleet Manager	Unique registration number	New vehicle in `available`

			status
UC-5: Register a Driver	Safety Officer	Unique license number, valid KYC	New driver in `available` status, PII encrypted
UC-6: Open Maintenance	Fleet Manager / Safety Officer	Vehicle not `on_trip` / `retired`	Vehicle enters maintenance workflow
UC-7: Close Maintenance	Fleet Manager / Safety Officer	Maintenance record `open`	Record `closed`
UC-8: Log Fuel/Expense	Financial Analyst / Fleet Manager	Vehicle exists	Cost data available for reporting
UC-9: View Reports/Export	Financial Analyst / Fleet Manager	—	Dashboard/report data or CSV returned
UC-10: Receive Maintenance Alert	Any user (broadcast)	Vehicle nears service mileage	Notification created/visible

---

6. Data Requirements

Full entity/attribute/validation detail is maintained in the **Backend Schema** document and visualized in the **ER Diagram**. Summary of persisted entities: `User`, `Vehicle`, `Driver`, `Trip`, `FuelLog`, `Expense`, `Maintenance`, `Notification`, `AuditLog`.

Key data-integrity requirements: - Unique constraints: `User.email`, `Vehicle.registrationNumber`, `Driver.licenseNumber`, `Trip.tripCode`. - Referential fields (`vehicle`, `driver`, `trip`, `userId`) are ObjectId references, validated for existence at the service/controller layer (Mongoose does not enforce FK integrity natively). - Soft-delete flag (`deleted`) must be respected by all read queries against base-schema collections.

---

7. Other Requirements / Known Constraints

- The AI route-optimization and AI-assistant endpoints are present as route stubs only and shall not be treated as functional requirements until implemented.
- The in-process response cache does not synchronize across multiple server instances; this is an accepted limitation for the current single-node-oriented deployment and should be revisited before horizontal scaling (see TRD §6).
- Two known schema/validation inconsistencies exist and should be resolved in a future maintenance pass (documented in full in the Backend Schema doc, §7): (a) Driver `status` enum mismatch between the Zod validator and the Mongoose schema (`on_leave` vs `off_duty`); (b) `constants/index.js` role list does not match the roles actually enforced in route middleware.
- No automated test suite currently exists in the repository; CI lint/test steps run conditionally (`--if-present`) and are effectively no-ops until tests are added.

---

8. Traceability Note

Every functional requirement in Section 3.1 was verified directly against the corresponding controller/service/route source file at authoring time (see PRD §5–6 and TRD §5 for the underlying business-rule and endpoint mapping) rather than being asserted independently of the implementation.

End of SRS. Companion documents: PRD, TRD, Backend Schema, ER Diagram, UML Class Diagram, Trip Lifecycle Workflow, UI/UX Theme Guide.